Universidad Mariano Gálvez de Guatemala
Boca del Monte

Ingeniería en Sistemas. Ciclo VIII
Jornada Sábado.
ANÁLISIS DE SISTEMAS | SEC 4
JUAN ALFONSO SANTIZO

Sistema de Transferencias Financieras

Información del documento

Campo	Detalle
Título del documento	Sistema de Transferencias Financieras
Proyecto	Sistema de Transferencias Financieras
Curso	Análisis de Sistemas II
Versión actual	1.0
Estado	Entregable final
Fecha de emisión	15 de octubre de 2026
Clasificación	Uso académico

Control de versiones

Versión	Fecha	Autor(es)	Descripción del cambio
1.0	15/10/2026	Equipo	Unificación, revisión de coherencia y versión final para entrega.

Equipo Responsable

Nombre	Rol / Responsabilidad	Carné
Astrid Noemi Herrera Solórzano	Requerimientos y Casos de Uso	3490-18-22721
Glenda Fabiola Alvarez Osoy	Arquitectura y Diseño de Componentes	3190-20-1940
Wuili Gustavo Sumale Pacheco	Modelado UML	7691-22-11007
Josué Fernando García Reyes	UX y Supuestos	9960-20-7708
Luis Fernando Lima Ixcuna	Pruebas y Trazabilidad	7690-20-17409

Índice

Requerimientos Funcionales **5**
Propósito 5
Requerimientos Funcionales 5
Reglas de negocio asociadas 6
Trazabilidad hacia casos de uso 7
Alcance y exclusiones 7
Requerimientos No Funcionales **8**
Propósito 8
Catálogo de Requerimientos No Funcionales 8
Desarrollo de los atributos críticos 9
Trazabilidad RNF 10
Alcance y exclusiones 10
Casos de uso **11**
Diagrama de casos de uso 12
Descripción extendida de los casos de uso 13
Decisión Arquitectónica **16**
Arquitectura 17
Justificación 17
Descripción de capas 18
Servicios 18
Justificación del modelo de cómputo 21
Restricciones que condicionaron la decisión 22
Propuesta alternativa 22
Alternativas Arquitectónicas Descartadas **25**
Microservicios 25
Arquitectura Hexagonal 26
Diseño de Componentes **27**
Catálogo de componentes 27
Relación entre componentes 28
Justificación de cohesión y bajo acoplamiento 29
Diagrama de Componentes **30**
Notas de diseño 31
Diagrama de Clases **31**
Descripción de las clases 31
Relación de las clases 32
Diagrama de clases 32
Diagrama de Secuencias **33**
Transacción exitosa 33

Transacción rechazada 34
Transacción duplicada 34
Observaciones 35
Supuestos del sistema **35**
Flujos UX **35**
Wireframes **35**
Casos de Pruebas **36**
Criterios de Aceptación **40**
Trazabilidad **43**
Matriz principal 43
Trazabilidad de requerimientos no funcionales 44
Cobertura de casos de uso 45
Verificación de integridad de la trazabilidad 45

Requerimientos Funcionales

Propósito

Este documento define, de manera concreta y verificable, las acciones que el Sistema de Transferencias Financieras debe permitir realizar. Expande la lista base del planteamiento del problema con el análisis de las deficiencias identificadas en el sistema actual: inconsistencias de saldos, falta de validación, operaciones duplicadas o incompletas, ausencia de registro y dificultad de auditoría.

Cada requerimiento se redacta de forma que pueda comprobarse mediante un caso de prueba, lo que habilita la trazabilidad obligatoria Requerimiento → Caso de uso → Diseño → Prueba.

Requerimientos Funcionales

Código	Requerimiento	Comportamiento esperado	Prioridad
RF-01	Crear cuentas	Registrar una cuenta nueva y asignarle un identificador único. Si el identificador lo genera el sistema, debe garantizarse su unicidad.	Alta
RF-02	Consultar cuentas	Verificar si una cuenta existe y consultar sus datos autorizados, estado y saldo disponible.	Alta
RF-03	Solicitar transferencias	Permitir indicar cuenta de origen, cuenta de destino y monto a transferir, junto con una referencia única de solicitud.	Alta
RF-04	Validar transferencias	Comprobar existencia y habilitación de ambas cuentas, que sean distintas, que el monto sea válido (> 0) y que existan fondos suficientes en el origen.	Crítica
RF-05	Evitar duplicados	Identificar solicitudes repetidas por su referencia única y evitar que una misma operación se ejecute dos veces.	Crítica
RF-06	Procesar transferencias	Descontar el monto de la cuenta de origen y acreditarlo en la cuenta de destino como una sola operación atómica.	Crítica

RF-07	Registrar transacciones	Guardar referencia única, fecha, monto, cuentas involucradas y resultado de cada operación (completada o rechazada).	Alta
RF-08	Consultar historial	Permitir consultar transferencias anteriores según los permisos del usuario, con filtros por cuenta y período.	Media
RF-09	Gestionar errores	Informar el motivo por el que una operación fue rechazada o no pudo completarse, sin dejar saldos inconsistentes.	Crítica

Reglas de negocio asociadas

Las reglas de negocio son la base que hace confiables las operaciones financieras. Cada regla respalda uno o más requerimientos funcionales.

Código	Regla de negocio	RF relacionados
RB-01	Identificación: cada cuenta y cada transferencia tendrán una referencia única.	RF-01, RF-03, RF-07
RB-02	Fondos suficientes: no se permitirá transferir un monto superior al saldo disponible.	RF-04
RB-03	Cuentas válidas: origen y destino deben existir y estar habilitadas para operar.	RF-04
RB-04	Integridad financiera: el monto debitado debe ser exactamente igual al monto acreditado.	RF-06
RB-05	Procesamiento único: repetir una solicitud no genera una segunda transferencia; la referencia única reconoce los reintentos.	RF-05
RB-06	Registro trazable: cada transferencia conserva su resultado, incluida la causa del rechazo cuando corresponda.	RF-07, RF-09
RB-07	Operación atómica: débito y crédito se confirman juntos o se cancelan juntos; ante un error no queda modificación parcial.	RF-06, RF-09

Trazabilidad hacia casos de uso

Los requerimientos funcionales se verán ejercidos por los casos de uso del sistema

RF	Caso(s) de uso que lo ejercen
RF-01	CU-01 Crear una cuenta
RF-02	CU-01, CU-02 (verificación previa)
RF-03, RF-04, RF-06	CU-02 Realizar una transferencia
RF-04, RF-05, RF-09	CU-03 Rechazar transferencia inválida o duplicada
RF-07	CU-02, CU-03 (registro del resultado)
RF-08	CU-04 Consultar el historial de transacciones

Alcance y exclusiones

- Dentro de alcance (fase 1): crear cuentas, validar y procesar transferencias, registrar operaciones y consultar historial.
- Fuera de alcance (fase 1): integración con bancos externos, pasarelas de pago, servicios antifraude de terceros, cambio de divisas y migración de datos. Estos aspectos se abordarán en fases posteriores.

Requerimientos No Funcionales

Propósito

Los requerimientos no funcionales (RNF) establecen qué tan confiable, seguro, íntegro y eficiente debe ser el sistema, más allá de las funciones que ejecuta. En un sistema financiero son determinantes: no basta con que una transferencia parezca funcionar, hay que demostrar que el dinero se conserva, que los errores se controlan y que las operaciones se pueden auditar.

Catálogo de Requerimientos No Funcionales

Código	Atributo de calidad	Criterio propuesto de aceptación	Prioridad
RNF-01	Consistencia financiera	En ninguna prueba se permitirá un débito sin su crédito correspondiente, ni un crédito sin su débito. El total de dinero del sistema no cambia por efecto de una transferencia.	Crítica
RNF-02	Confiabilidad	Ante fallos simulados durante una transferencia, los saldos permanecen consistentes y el resultado de la operación siempre puede determinarse (completada o rechazada, nunca ambiguo).	Crítica
RNF-03	Seguridad	Solo usuarios autenticados y autorizados pueden ejecutar las acciones correspondientes a sus permisos.	Alta
RNF-04	Integridad y trazabilidad	Cada operación tiene una referencia única y un registro consultable con su estado y datos relevantes.	Alta
RNF-05	Rendimiento	Definir con el cliente un tiempo máximo de respuesta y una carga de operaciones esperada; validar ambos mediante pruebas. (Objetivo a acordar)	Media
RNF-06	Disponibilidad	Acordar un objetivo de disponibilidad y un procedimiento de recuperación ante interrupciones. (Objetivo a acordar)	Media
RNF-07	Mantenibilidad	Las reglas de validación deben poder modificarse sin obligar a rediseñar todo el sistema.	Alta

RNF-08	Privacidad	El historial solo muestra información financiera a usuarios con permisos adecuados.	Alta

Desarrollo de los atributos críticos

Los tres primeros RNF son los que sostienen la confianza en un sistema financiero y merecen desarrollo detallado.

RNF-01 · Consistencia financiera (atomicidad)

El débito y el crédito forman una transacción atómica: ambos se confirman o ninguno se aplica (regla de negocio RB-07). Si falla cualquier paso tras el débito, el sistema revierte el cambio (rollback) y deja los saldos como estaban. El invariante es: la suma total de saldos no se altera por una transferencia; solo se redistribuye.

RNF-02 · Confiabilidad (manejo de errores y resultado determinable)

Toda operación termina en un estado explícito y verificable (COMPLETADA o RECHAZADA), nunca en un estado ambiguo. Ante una interrupción, debe existir un procedimiento para determinar el resultado real antes de permitir un reintento, evitando dobles débitos. Esto se apoya en el registro trazable (RB-06) y la referencia única de solicitud (RB-05).

RNF-03 · Seguridad (autenticación y autorización)

El acceso a funciones restringidas exige identificación previa. Las acciones se limitan según el rol del usuario: administrar cuentas, solicitar/procesar transferencias o consultar historial son permisos diferenciados. Los permisos definitivos se confirman con la institución.

Trazabilidad RNF

RNF	Se apoya en RF	Reglas de negocio
RNF-01 Consistencia	RF-06	RB-04, RB-07
RNF-02 Confiabilidad	RF-05, RF-06, RF-09	RB-05, RB-06, RB-07
RNF-03 Seguridad	RF-02, RF-08	—
RNF-04 Integridad y trazabilidad	RF-03, RF-07	RB-01, RB-06
RNF-05 Rendimiento	RF-06	—
RNF-06 Disponibilidad	RF-09	RB-07
RNF-07 Mantenibilidad	RF-04	—

Alcance y exclusiones

Estos requerimientos permiten evaluar la propuesta con evidencias:

- Consistencia y confiabilidad → el dinero se conserva incluso ante fallos.
- Seguridad y privacidad → solo quien debe, ve y opera lo que le corresponde.
- Integridad y trazabilidad → toda operación es auditable.
- Mantenibilidad → las reglas de negocio evolucionan sin reconstruir el sistema.

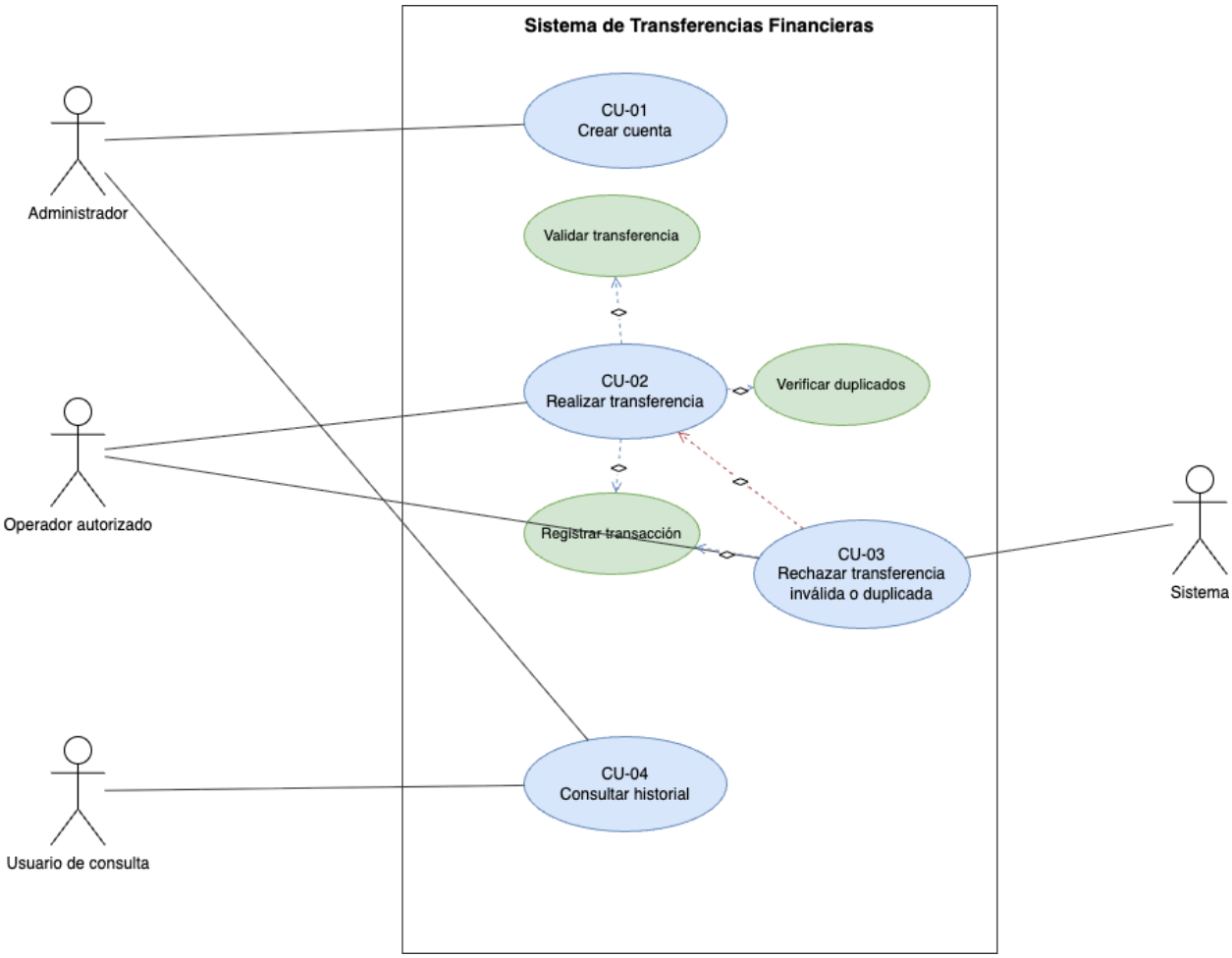
Casos de uso

Los casos de uso describen cómo interactúan los usuarios con el sistema para alcanzar un objetivo concreto. Permiten visualizar qué podrá hacer cada rol y qué respuesta recibirá ante situaciones normales, alternas o de error. Cada caso de uso está ligado a los requerimientos funcionales que ejercita, sosteniendo la trazabilidad Requerimiento → Caso de uso → Diseño → Prueba.

Actores del sistema

Actor	Descripción	Permisos principales
Administrador	Registra cuentas y consulta la información que sus permisos le permitan.	Crear y consultar cuentas.
Operador autorizado	Solicita o procesa transferencias conforme a los permisos establecidos.	Solicitar y procesar transferencias.
Usuario de consulta	Revisa el historial de operaciones que tiene permitido consultar.	Consultar historial.

Diagrama de casos de uso



Listado de casos de uso

ID	Caso de uso	Actor principal	RF relacionados
CU-01	Crear una cuenta	Administrador	RF-01, RF-02
CU-02	Realizar una transferencia	Operador autorizado	RF-03, RF-04, RF-06, RF-07
CU-03	Rechazar transferencia inválida o duplicada	Operador autorizado	RF-04, RF-05, RF-09
CU-04	Consultar el historial de transacciones	Usuario de consulta	RF-08

Descripción extendida de los casos de uso

CU-01 · Crear una cuenta

Actor: Administrador

Objetivo: Registrar una nueva cuenta en el sistema.

Precondiciones: El administrador está autenticado y autorizado.

Postcondiciones: La cuenta queda registrada con un identificador único.

Flujo principal

1. El administrador solicita crear una cuenta.
2. El sistema verifica los datos obligatorios y que no exista una cuenta con el mismo identificador.
3. Si el identificador lo genera el sistema, esté asigna uno único.
4. La cuenta se registra y el sistema confirma su creación.

Flujo de error — identificador duplicado

En el paso 2, si ya existe una cuenta con el mismo identificador, la operación se rechaza y la cuenta original permanece sin cambios.

Criterio de aceptación

No pueden existir dos cuentas con el mismo identificador. Una cuenta solo se confirma como creada cuando el registro se completa correctamente.

CU-02 · Realizar una transferencia

Actor: Operador autorizado

Objetivo: Mover fondos entre dos cuentas.

Precondiciones: Operador autenticado; cuentas origen y destino existentes y habilitadas.

Postcondiciones: Saldos actualizados y transacción registrada con referencia única.

Flujo principal

1. El operador ingresa cuenta de origen, cuenta de destino y monto.
2. El sistema verifica que ambas cuentas existan, estén habilitadas y sean distintas, y que el monto sea válido (> 0).
3. El sistema comprueba que la cuenta de origen disponga de fondos suficientes.
4. El sistema verifica que la solicitud no sea un duplicado (por su referencia única).
5. El sistema descubre los fondos del origen y los acredita en el destino como una única operación atómica.
6. El sistema registra la transacción con su referencia única y confirma el resultado al operador.

Flujo alterno — fondos insuficientes

En el paso 3, si no hay fondos suficientes, el sistema rechaza la operación, conserva los saldos originales y deriva a CU-03.

Flujo de error — fallo durante el proceso

Si ocurre un error entre el débito y el crédito (paso 5), el sistema revierte el débito (rollback) para no dejar una operación parcial, y registra el resultado como rechazado.

Criterio de aceptación

El monto debitado del origen es exactamente igual al acreditado en destino. Si la operación no puede completarse íntegramente, ningún saldo se modifica.

CU-03 · Rechazar una transferencia inválida o duplicada

Actor: Operador autorizado

Objetivo: Impedir movimientos incorrectos o repetidos.

Precondiciones: Existe una solicitud de transferencia en proceso de validación.

Postcondiciones: La operación queda rechazada y registrada; ningún saldo se modifica.

Escenarios de rechazo

- Fondos insuficientes: se rechazan y se conservan los saldos originales.
- Cuenta inexistente o inhabilitada: se informa que la operación no puede realizarse.
- Monto inválido: se rechazan importes ≤ 0 y cualquiera que incumpla las reglas acordadas.
- Solicitud duplicada: si se recibe de nuevo la misma referencia de solicitud, el sistema no vuelve a ejecutar la transferencia y devuelve el resultado original o su estado verificable.
- Error durante el proceso: se evita confirmar una operación parcial y se verifica su resultado antes de permitir un nuevo intento.

Criterio de aceptación

Ninguna solicitud rechazada modifica los saldos. Reintentar una solicitud ya procesada no genera un segundo débito ni un segundo crédito.

CU-04 · Consultar el historial de transacciones

Actor: Usuario de consulta

Objetivo: Revisar las operaciones realizadas.

Precondiciones: Usuario autenticado con permisos de consulta.

Postcondiciones: Se muestran las operaciones permitidas según los filtros indicados.

Flujo principal

El usuario accede a la consulta de historial.

Indica los criterios disponibles (por ejemplo, cuenta y período).

El sistema verifica sus permisos y presenta las operaciones correspondientes.

El usuario consulta el detalle y el estado de una transacción.

Flujo alterno — sin resultados

Si no existen transacciones para los criterios indicados, el sistema muestra un resultado vacío informando que no hay operaciones.

Ejemplo práctico

Un responsable de operaciones busca las transferencias de una cuenta del 14 de octubre. Encuentra una transferencia por Q300.00, identifica sus cuentas de origen y destino, consulta su referencia y comprueba que fue completada.

Criterio de aceptación

El historial muestra información coherente con los registros originales, distingue operaciones completadas de rechazadas y respeta los permisos de consulta.

Decisión Arquitectónica

La institución requiere un sistema de transferencias confiable cuyo atributo de calidad dominante es la consistencia financiera: ninguna operación puede crear ni destruir dinero. A esto se suman la confiabilidad ante fallos, la trazabilidad de cada operación, la seguridad por rol y la mantenibilidad de las reglas de negocio.

Condiciones de despliegue que acotan la decisión:

- El sistema se desplegará en AWS (nube), no on-premise.
- La integración con sistemas internos de la institución (on-premise) se realizará mediante VPN sitio a sitio y servicios API REST / SOAP.
- La primera fase cubre creación de cuentas, validación y procesamiento de transferencias, registro y consulta de historial; no contempla integración con bancos externos ni pasarelas de terceros.

Arquitectura

Arquitectura en capas con un núcleo de dominio transaccional, desplegada como aplicación modular contenedizada en AWS, e integrada con los sistemas on-premise a través de una capa anticorrupción (Anti-Corruption Layer).

El principio rector es: la operación de transferencia (débito + crédito) ocurre como una sola transacción ACID dentro de un único almacén de datos transaccional, evitando la consistencia distribuida.

Justificación

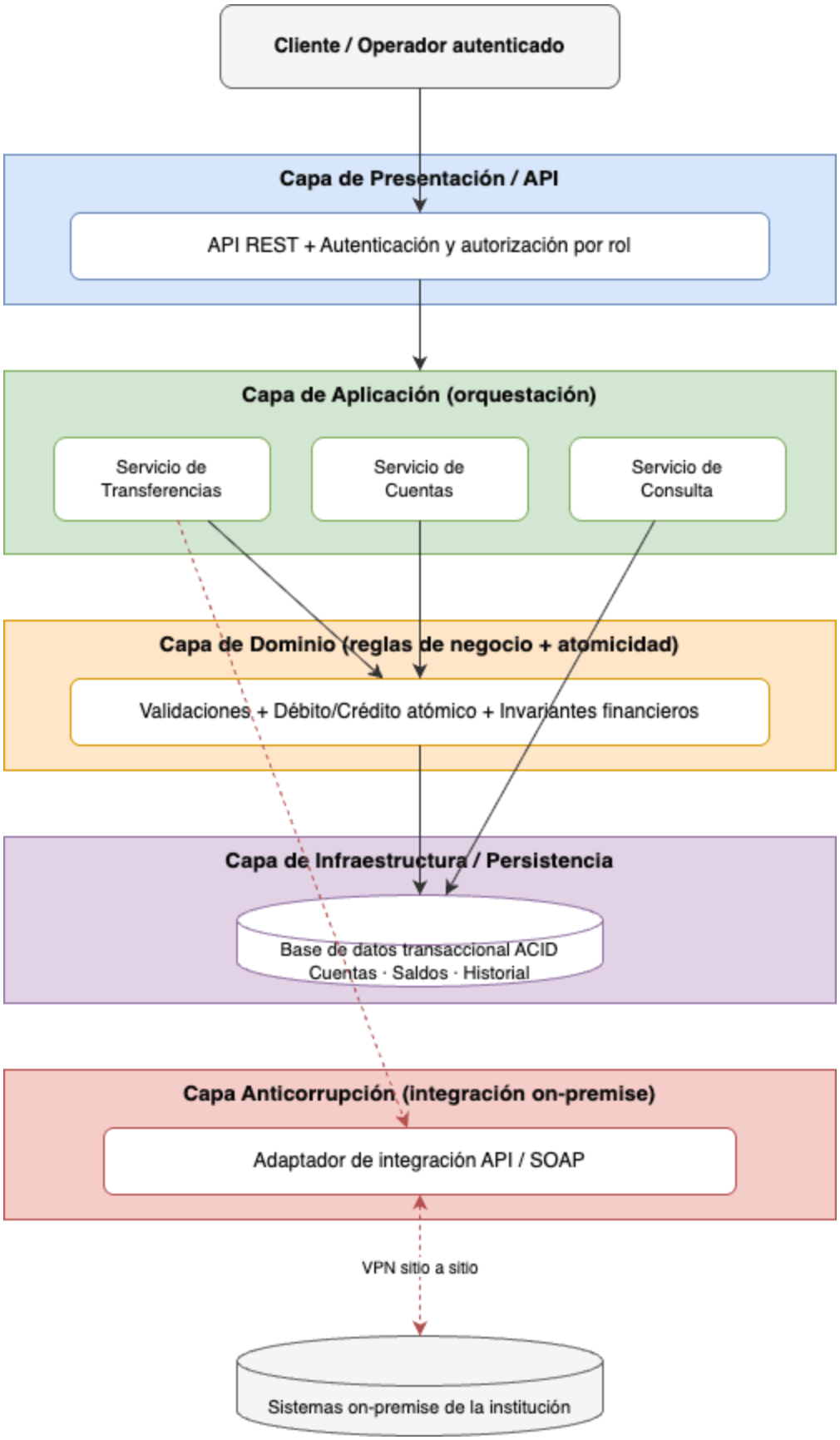
Requerimiento no funcional	**Cómo lo satisface la arquitectura elegida**
Consistencia (RNF-01)	El débito y el crédito se ejecutan en una transacción ACID única sobre una base de datos relacional gestionada. Si algo falla, se revierte todo (rollback), cumpliendo la regla de atomicidad (RB-07). No hay transacciones repartidas entre servicios.
Confiabilidad (RNF-02)	El resultado de toda operación queda determinado (COMPLETADA/RECHAZADA) y persistido antes de responder. La idempotencia por referencia única (RB-05) permite reintentos seguros ante cortes de red o VPN.
Seguridad (RNF-03)	La autenticación y autorización por rol se concentran en la capa de presentación/aplicación, apoyadas en servicios de identidad gestionados.
Integridad y trazabilidad (RNF-04)	El registro de transacciones vive en la capa de infraestructura con referencia única y estado consultable.
Mantenibilidad (RNF-07)	Las reglas de validación se aíslan en la capa de dominio; cambiarlas no obliga a tocar presentación, persistencia ni integraciones. La capa anticorrupción aísla el núcleo de los detalles de SOAP/VPN.

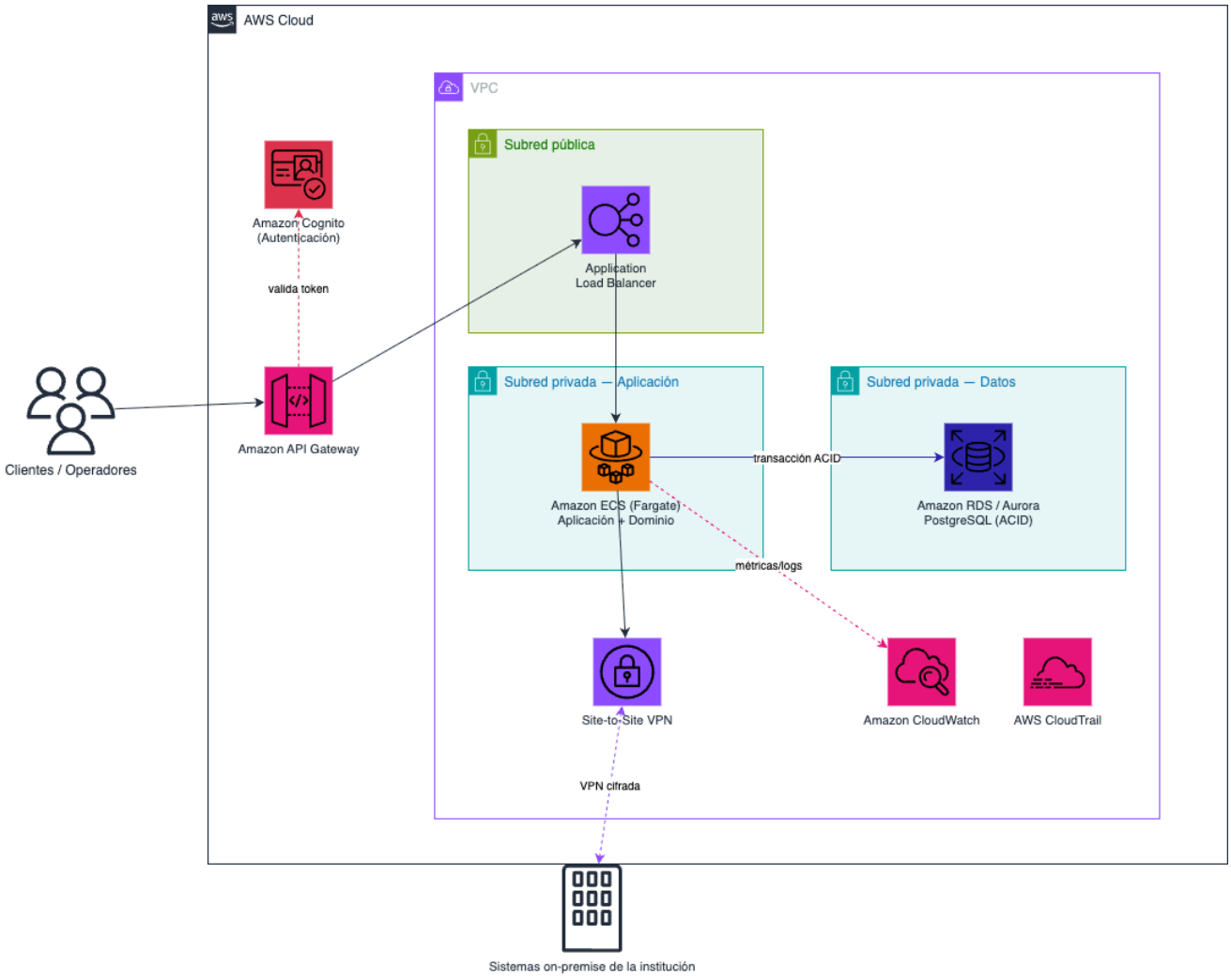
Descripción de capas

Capa / Módulo	Responsabilidad	Componentes del sistema
Presentación / API	Exponer la API a los clientes, autenticar y autorizar.	API pública REST, control de acceso por rol.
Aplicación (orquestación)	Coordinar cada caso de uso, abrir y cerrar la transacción, controlar la idempotencia.	Servicio de transferencias, servicio de cuentas, servicio de consulta.
Dominio (reglas de negocio)	Ejecutar validaciones y la operación atómica débito/crédito. Es el corazón del sistema.	Validaciones, lógica de débito/crédito, invariantes financieros.
Infraestructura / persistencia	Guardar cuentas, saldos e historial con garantías ACID.	Repositorio de cuentas, repositorio de historial.
Integración (capa anticorrupción)	Traducir entre el dominio y los sistemas on-premise vía VPN + API/SOAP, sin contaminar el núcleo.	Adaptador de integración on-premise.

Servicios

Capa lógica	Servicio AWS propuesto	Justificación
Presentación / API	API Gateway + Application Load Balancer	Punto de entrada gestionado, con control de tráfico y TLS.
Identidad y acceso	Amazon Cognito + IAM	Autenticación de usuarios y autorización por rol (RNF-03).
Aplicación y dominio	Amazon ECS (Fargate)	Contenedores sin gestión de servidores; la lógica de dominio corre aquí.
Persistencia	Amazon RDS / Aurora PostgreSQL	Transacciones ACID nativas que garantizan atomicidad débito/crédito (RNF-01).
Idempotencia / reintentos	Tabla de referencias de solicitud en RDS	Soporta RB-05 (procesamiento único) ante reintentos por cortes de VPN.
Integración on-premise	AWS Site-to-Site VPN + adaptador API/SOAP	Canal cifrado hacia los sistemas internos de la institución.
Observabilidad y auditoría	Amazon CloudWatch + AWS CloudTrail	Trazabilidad operativa y de accesos (apoya RNF-04).


Cliente / Operador autenticado
Capa de Presentación / API
API REST + Autenticación y autorización por rol
Capa de Aplicación (orquestación)
Servicio de Transferencias
Servicio de Cuentas
Servicio de Consulta
Capa de Dominio (reglas de negocio + atomicidad)
Validaciones + Débito/Crédito atómico + Invariantes financieros
Capa de Infraestructura / Persistencia
Base de datos transaccional ACID
Cuentas · Saldos · Historial
Capa Anticorrupción (integración on-premise)
Adaptador de integración API / SOAP
VPN sitio a sitio
Sistemas on-premise de la institución


AWS Cloud
VPC
Subred pública
Application
Load Balancer
Amazon Cognito
(Autenticación)
valida token
Clientes / Operadores
Amazon API Gateway
Subred privada — Aplicación
Subred privada — Datos
transacción ACID
Amazon ECS (Fargate)
Aplicación + Dominio
Amazon RDS / Aurora
PostgreSQL (ACID)
métricas/logs
Site-to-Site VPN
Amazon CloudWatch
AWS CloudTrail
VPN cifrada
Sistemas on-premise de la institución

La elección de una base de datos relacional transaccional (RDS/Aurora) sobre una NoSQL es deliberada: el invariante financiero exige transacciones ACID multi-registro, que las bases relacionales ofrecen de forma nativa.

Justificación del modelo de cómputo

La capa de aplicación y dominio se ejecuta en contenedores gestionados con Amazon ECS sobre Fargate. Esta elección se comparó explícitamente contra las otras dos opciones de cómputo de AWS:

Cómputo	Implicación	Justificación
EC2 (servidores virtuales)	El equipo administra el sistema operativo, parches, escalado, hardening y disponibilidad de las instancias.	Descartada como opción principal. Traslada al equipo la carga operativa de parchear y asegurar servidores —algo especialmente sensible en un sistema financiero— sin aportar ventaja funcional. Aumenta la superficie de responsabilidad en seguridad (RNF-03) y el esfuerzo de mantenimiento (RNF-07). Solo se justificaría si se necesitara control fino del SO o software que exija instalación a nivel de host, lo cual no ocurre aquí.
AWS Lambda (serverless por función)	Ejecución por evento, sin servidores, escalado automático hasta cero.	Descartada para el núcleo transaccional. El modelo de función efímera complica mantener conexiones persistentes y transacciones de larga vida contra RDS (agotamiento del connection pool bajo concurrencia, cold starts que afectan la latencia de una operación crítica). La transacción ACID débito/crédito se modela con más naturalidad en un proceso de larga vida.
ECS Fargate (contenedores gestionados)	Se despliega el contenedor; AWS gestiona el sustrato (no hay servidores que administrar).	Elegida. Elimina la administración de servidores (sin parcheo de SO, sin hardening manual), mantiene un proceso de larga vida ideal para transacciones ACID y connection pooling estable contra RDS, y escala por tareas según demanda. Ofrece el equilibrio óptimo entre control y carga operativa para este caso.

Resumen del criterio: EC2 da demasiada responsabilidad operativa (riesgo y mantenimiento), Lambda no encaja con transacciones de base de datos de larga vida, y Fargate ofrece el punto medio: contenedores portables y controlables, sin administrar servidores. Además, al estar la lógica contenedizada, el mismo contenedor puede ejecutarse on-premise, lo que preserva la portabilidad de la solución.

Restricciones que condicionaron la decisión

- Alcance acotado a la fase 1: no se justifica la complejidad operativa de microservicios o de consistencia distribuida.
- Despliegue en nube (AWS): permite usar servicios gestionados para persistencia, identidad y red, reduciendo el esfuerzo de operación.
- Integración on-premise por VPN + API/SOAP: obliga a aislar el núcleo mediante una capa anticorrupción, evitando que los detalles del legacy permeen al dominio.
- Evaluación por profundidad de diseño: se prioriza un estilo claro, justificable y defendible sobre uno vistoso pero sobredimensionado.

Propuesta alternativa

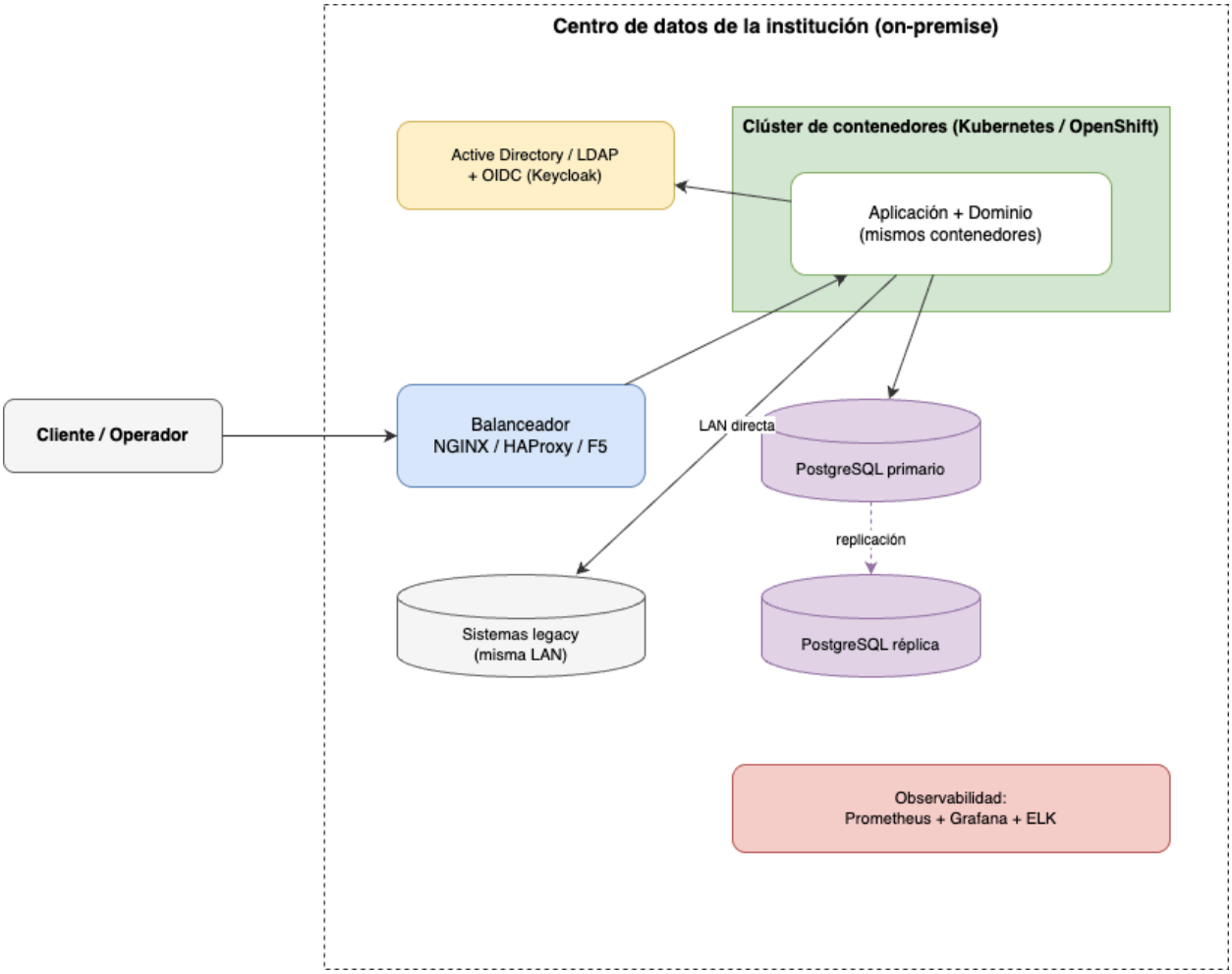
El mismo estilo arquitectónico (Capas con núcleo de dominio transaccional) es tecnológicamente neutro y puede desplegarse on-premise, en el centro de datos de la institución. Esto es posible precisamente porque la lógica va contenerizada y el dominio no depende de servicios propietarios de AWS.

Capa lógica	Equivalente on-premise
Presentación / API	Balanceador propio (NGINX / HAProxy / F5) + gateway de API self-hosted
Identidad y acceso	Directorio corporativo (Active Directory / LDAP) + Keycloak u OIDC self-hosted
Aplicación y dominio	Contenedores sobre Kubernetes (o OpenShift) en servidores propios, o VMs con el runtime de contenedores
Persistencia	PostgreSQL en clúster (primario + réplica) sobre hardware propio, con almacenamiento redundante
Integración	Red LAN interna directa a los sistemas legacy (sin necesidad de VPN, al estar en la misma red)
Observabilidad	Prometheus + Grafana + stack de logs (ELK) self-hosted

Comparativa

Criterio	AWS (recomendado)	On-premise
Carga operativa (parcheo, hardening, disponibilidad)	Gestionada por el proveedor	Responsabilidad total del equipo
Tiempo para estar productivo	Rápido (servicios listos)	Lento (aprovisionar hardware, instalar, configurar)
Alta disponibilidad y recuperación ante desastres	Multi-AZ nativo	Requiere segundo sitio y diseño propio
Escalado ante picos de transacciones	Elástico, bajo demanda	Limitado por el hardware adquirido
Inversión inicial (CAPEX)	Baja (pago por uso)	Alta (compra de hardware)
Control físico y soberanía del dato	Menor	Total
Integración con legacy on-premise	Vía VPN + API/SOAP (capa anticorrupción)	Directa en LAN

Conclusión: para la fase 1 y el perfil de este proyecto, AWS es la opción recomendada porque reduce la carga operativa y de seguridad, acelera la puesta en marcha y ofrece alta disponibilidad sin inversión inicial en hardware. El on-premise no se descarta como mala arquitectura —de hecho es la misma— sino que se reserva como alternativa justificable si pesan requisitos regulatorios de soberanía de datos o si la institución ya tiene la capacidad operativa instalada. Gracias a la contenerización, migrar entre ambos mundos es de bajo costo: el núcleo del sistema no cambia.



Alternativas Arquitectónicas Descartadas

El proyecto exige comparar la arquitectura elegida (Capas con núcleo de dominio transaccional sobre AWS) contra al menos dos estilos alternativos, justificando por qué fueron descartados para este problema y este alcance. El criterio de comparación principal es siempre el mismo: cómo cada estilo garantiza la consistencia financiera (no perder ni crear dinero) bajo despliegue en AWS con integración on-premise vía VPN + API/SOAP.

Microservicios

Descripción breve: El sistema se dividiría en servicios independientes y desplegables por separado (por ejemplo: servicio de cuentas, servicio de transferencias, servicio de historial), cada uno con su propia base de datos, comunicándose por red.

Ventajas para este problema:

- Escalado independiente de cada servicio.
- Despliegues parciales sin afectar a todo el sistema.
- Buen encaje natural con servicios gestionados de AWS.

Desventajas / razones de descarte:

- Rompe la atomicidad del débito/crédito: si "cuentas" y "transferencias" viven en bases de datos separadas, una transferencia deja de ser una transacción ACID única y exige consistencia distribuida (patrón Saga, compensaciones o two-phase commit).
- Eso introduce estados intermedios en los que el dinero puede quedar temporalmente "en el aire", justo el riesgo que el sistema debe eliminar (RNF-01, RB-07).
- La complejidad operativa (orquestación, mensajería, monitoreo distribuido) es desproporcionada para el alcance de la fase 1.

- Razón específica de descarte: garantizar que nunca exista un débito sin su crédito es mucho más simple y robusto con una única transacción ACID que coordinando compensaciones entre servicios.

Arquitectura Hexagonal

Descripción breve: El dominio queda totalmente aislado en el centro y toda interacción externa (API, base de datos, integraciones) entra y sale por "puertos" implementados por "adaptadores". Es un estilo excelente para aislar la lógica de negocio de la tecnología.

Ventajas para este problema:

- Máximo aislamiento del dominio financiero respecto de la tecnología (gran mantenibilidad y testabilidad).
- Facilita sustituir adaptadores (por ejemplo, cambiar el mecanismo de integración on-premise) sin tocar el núcleo.

Desventajas / razones de descarte:

- Para el alcance de la fase 1, la ceremonia de definir puertos y adaptadores para cada interacción añade complejidad conceptual que no se traduce en beneficio proporcional.
- El equipo y la institución defienden con más facilidad un modelo en capas clásico, que es ampliamente conocido y suficiente para los requerimientos actuales.
- Razón específica de descarte: no se descarta por ser mala (de hecho es una evolución natural), sino por sobredimensionada para esta fase. La arquitectura elegida ya incorpora su mejor idea —aislar el dominio— mediante la capa anticorrupción para las integraciones on-premise, sin pagar el costo de hexagonalizar todo el sistema.

Diseño de Componentes

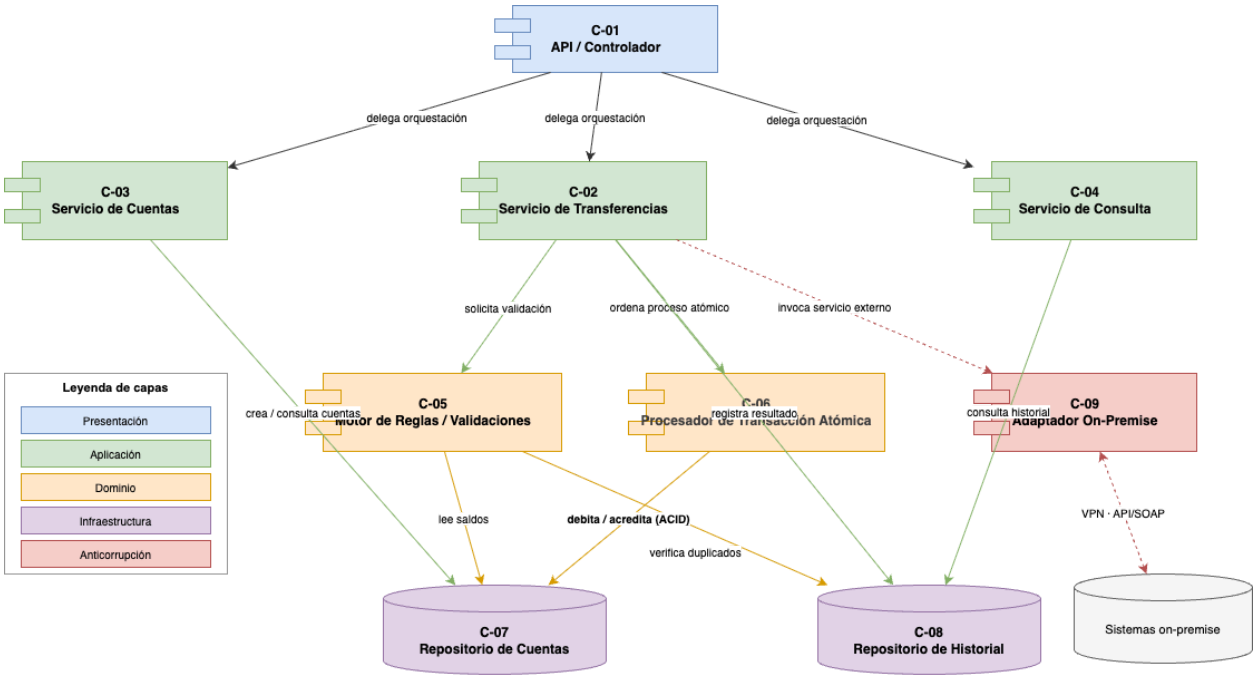
Objetivo: Dividir el sistema en componentes con responsabilidades claras, garantizando alta cohesión (cada componente tiene una única razón para cambiar) y bajo acoplamiento (los componentes dependen de interfaces, no de implementaciones). Cada componente se ubica en una de las cinco capas definidas en la decisión arquitectónica y es la base para el diagrama de componentes y el diagrama de clases.

Catálogo de componentes

ID	Componente	Capa	Responsabilidad	Interfaz expuesta
C-01	API / Controlador	Presentación	Recibir peticiones, autenticar/autorizar por rol y delegar en los servicios de aplicación.	crearCuenta(), transferir(), consultarHistorial()
C-02	Servicio de Transferencias	Aplicación	Orquestar el caso de uso de transferencia: abrir la transacción, coordinar validación, proceso atómico y registro, controlar la idempotencia por referencia única.	ejecutarTransferencia(solicitud)
C-03	Servicio de Cuentas	Aplicación	Orquestar la creación y consulta de cuentas.	crearCuenta(datos), consultarCuenta(id)
C-04	Servicio de Consulta	Aplicación	Orquestar la consulta de historial respetando permisos y filtros.	consultarHistorial(criterios)
C-05	Motor de Reglas / Validaciones	Dominio	Validar existencia y habilitación de cuentas, monto válido, fondos suficientes y duplicados. Concentra las reglas de negocio.	validarTransferencia(solicitud), esDuplicada(ref)
C-06	Procesador de Transacción Atómica	Dominio	Ejecutar débito en origen y crédito en destino como una sola operación ACID; revertir ante fallo.	procesar(origen, destino, monto)
C-07	Repositorio de Cuentas	Infraestructura	Persistir y recuperar cuentas y saldos con garantías transaccionales.	obtener(id), actualizarSaldo(id, saldo)
C-08	Repositorio de Historial	Infraestructura	Guardar y consultar el registro de transacciones con su estado.	guardar(transaccion), buscar(criterios)

C-09	Adaptador de Integración on-premise	Anticorrupción	Traducir entre el dominio y los sistemas on-premise vía API/SOAP sobre VPN, sin contaminar el núcleo.	invocarServicioExterno(peticion)

Relación entre componentes



Justificación de cohesión y bajo acoplamiento

Alta cohesión:

- Cada componente tiene una sola responsabilidad: C-05 solo valida, C-06 solo ejecuta el movimiento atómico, los repositorios (C-07, C-08) solo persisten. Ninguno mezcla responsabilidades de otra capa.
- Las reglas de negocio están aisladas en el dominio (C-05, C-06), de modo que un cambio en una regla no afecta a presentación ni a persistencia (sostiene la mantenibilidad, RNF-07).

Bajo acoplamiento:

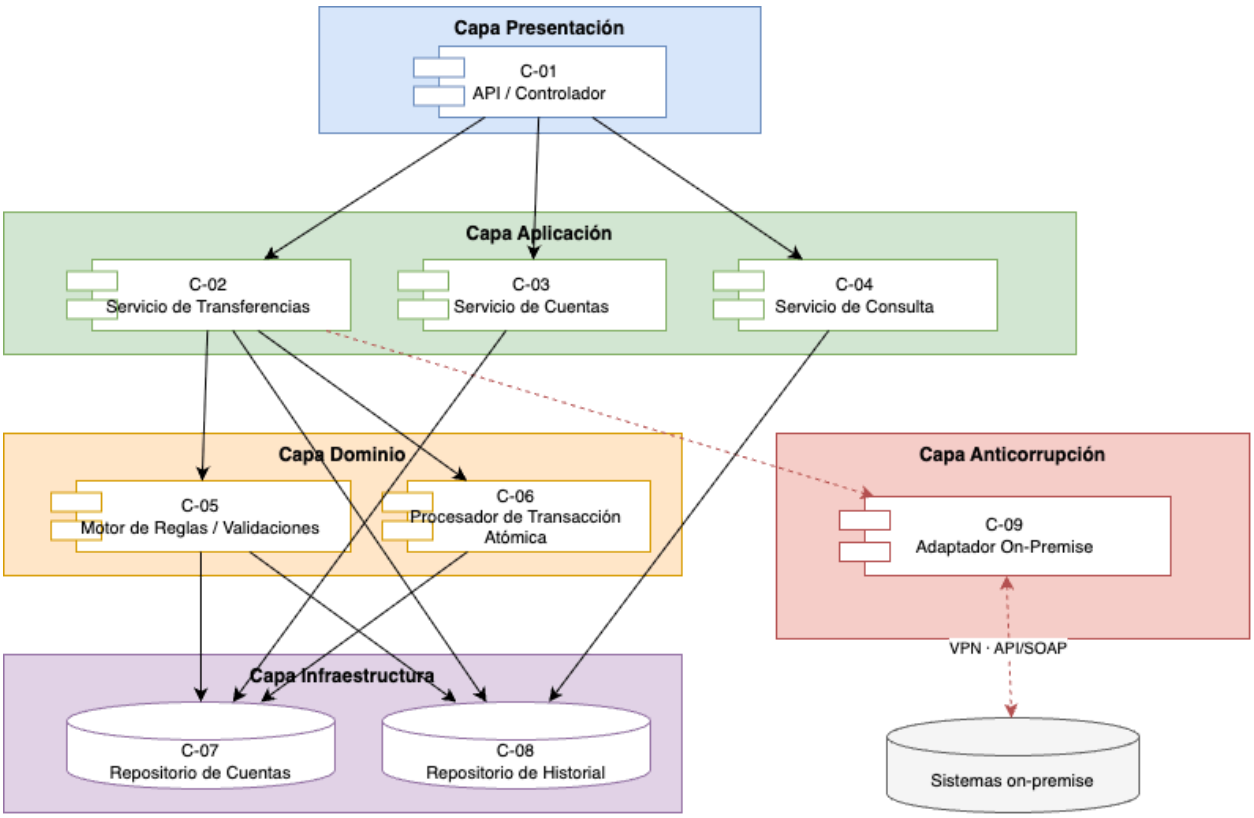
- Los servicios de aplicación dependen de las interfaces de los repositorios y del dominio, no de sus implementaciones. Cambiar la base de datos o el mecanismo de integración on-premise no obliga a tocar la lógica de negocio.
- El adaptador C-09 actúa como barrera: los detalles de SOAP/VPN quedan encapsulados y no permean al dominio (patrón anticorrupción).

Concentración del invariante crítico:

- Solo C-06 altera saldos y siempre actúa atómicamente. Esto hace que garantizar "ningún débito sin su crédito" (RNF-01, RB-04, RB-07) sea verificable en un único punto del diseño.

Diagrama de Componentes

Representar visualmente los componentes del sistema, sus interfaces y dependencias, agrupados por la capa arquitectónica a la que pertenecen. Los IDs coinciden con los definidos en el diseño de componentes.



Notas de diseño

- El flujo crítico (transferencia) atraviesa C-01 → C-02 → {C-05 validación, C-06 proceso atómico, C-08 registro}.
- C-06 es el único que escribe saldos (vía C-07), siempre dentro de una transacción ACID.
- C-09 aísla los detalles de la integración on-premise del resto del sistema (barrera anticorrupción).

Diagrama de Clases

Modelar las clases que implementan los componentes definidos en el diseño de componentes, con sus atributos, métodos y relaciones. El diseño concentra la modificación de saldos en una única clase (ProcesadorTransaccion) para proteger el invariante financiero.

Descripción de las clases

Clase	Componente	Responsabilidad
Cuenta	C-07	Representa una cuenta con saldo e identificador único; sabe debitar y acreditar.
Transaccion	C-08	Registra los datos y el resultado de cada operación.
EstadoTransaccion	—	Enumeración del estado: EXITOSA, RECHAZADA, PENDIENTE.
TransferenciaService	C-02	Orquesta la transferencia y controla la idempotencia.
CuentaService	C-03	Crea y consulta cuentas.
ConsultaService	C-04	Consulta el historial según permisos y filtros.
ValidadorFondos	C-05	Valida existencia, monto, fondos y duplicados.
ProcesadorTransaccion	C-06	Única clase que modifica saldos, siempre de forma atómica.
CuentaRepository	C-07	Persiste y recupera cuentas.

HistorialRepository	C-08	Persiste y consulta transacciones.

Relación de las clases

Relación	Tipo
TransferenciaService → ValidadorFondos	Dependencia
TransferenciaService → ProcesadorTransaccion	Dependencia
TransferenciaService → HistorialRepository	Dependencia
ProcesadorTransaccion → Cuenta	Asociación (modifica saldos)
ProcesadorTransaccion → Transaccion	Creación
Transaccion → EstadoTransaccion	Composición
CuentaRepository → Cuenta	Gestión / persistencia
HistorialRepository → Transaccion	Gestión / persistencia

Diagrama de clases

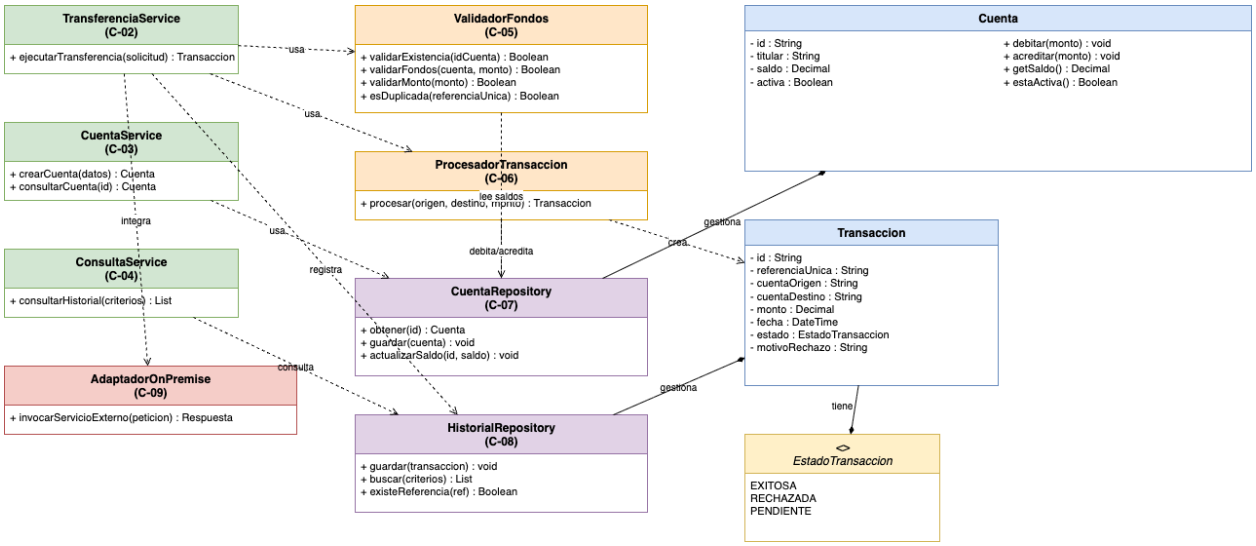
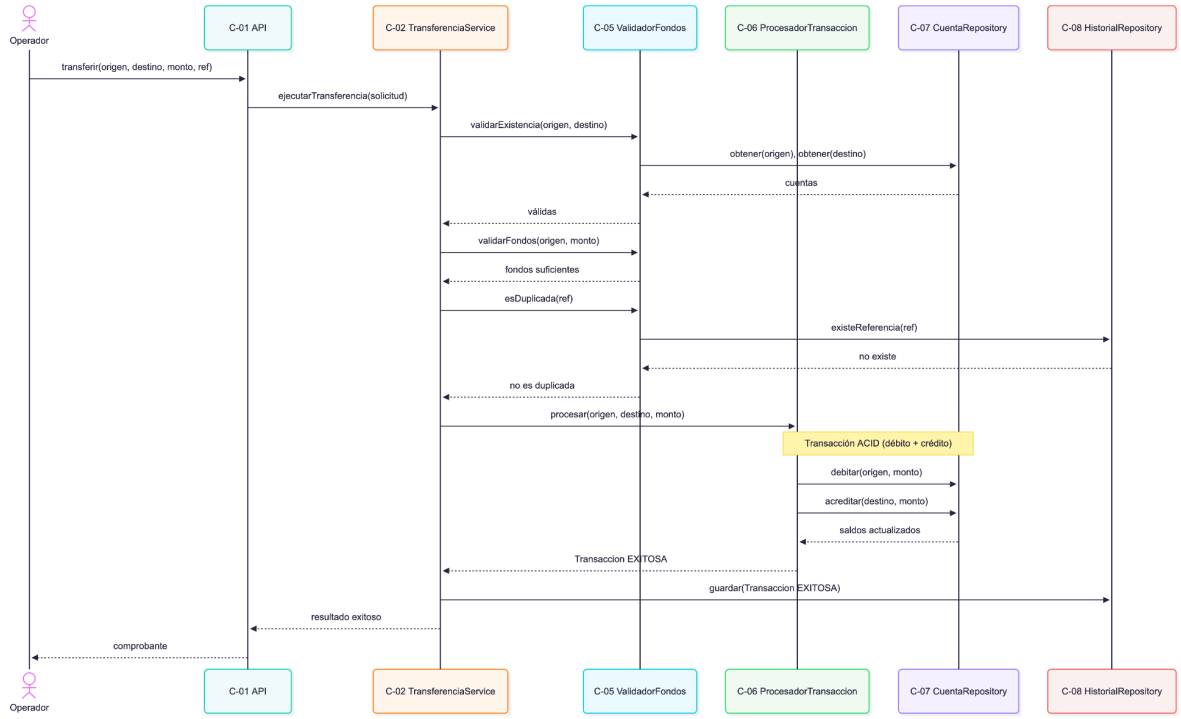


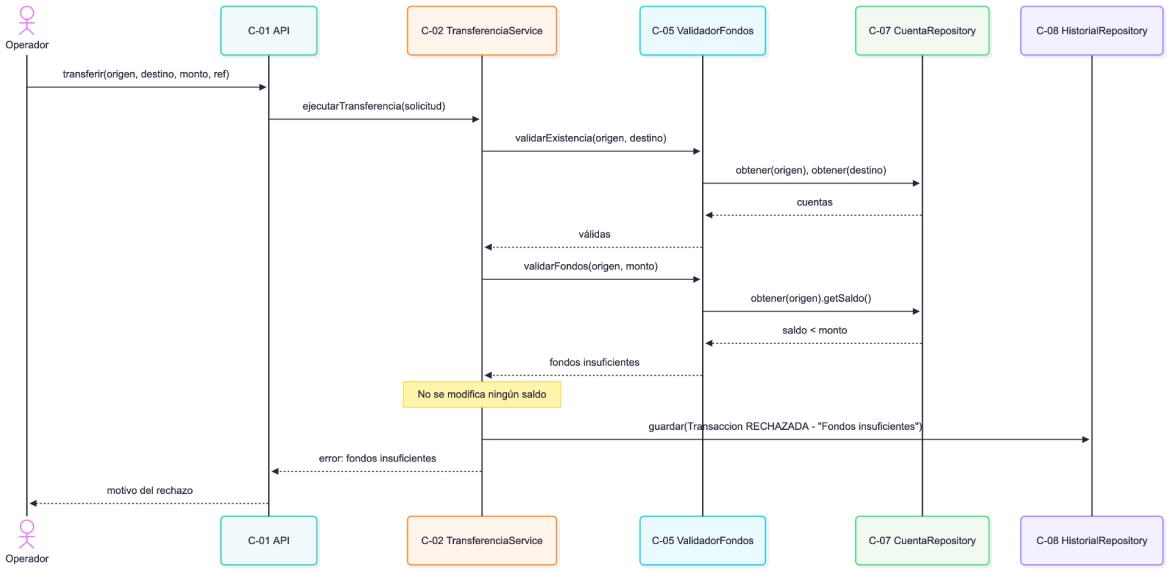
Diagrama de Secuencias

El propósito es mostrar la interacción temporal entre el actor y los componentes/clases del sistema para los flujos principales de la transferencia. Los participantes corresponden a las clases del diagrama de clases y a los componentes C-01..C-09.

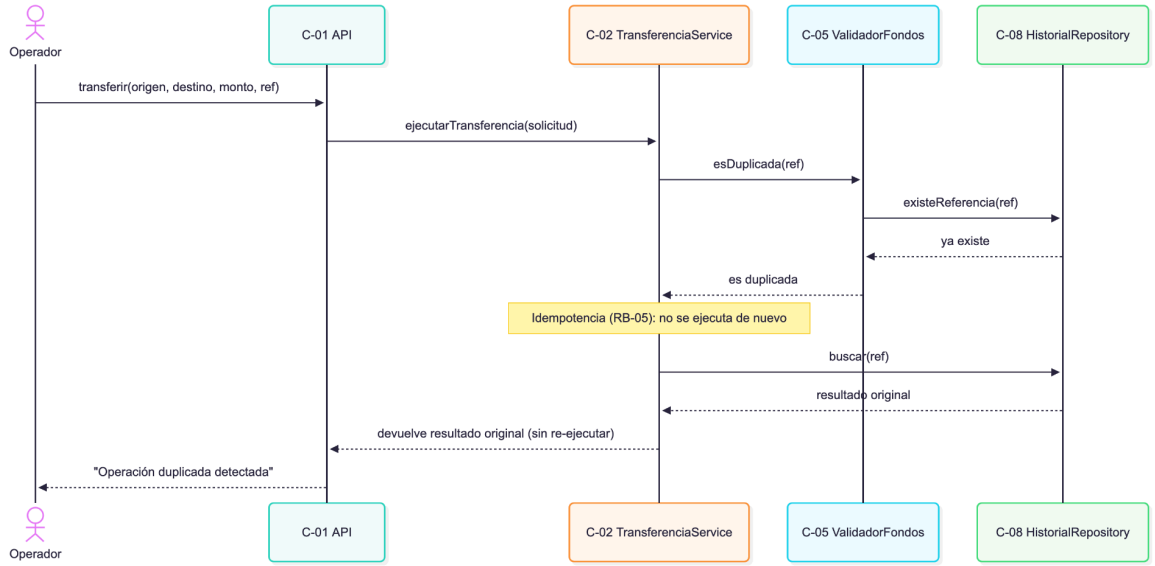
Transacción exitosa



Transacción rechazada



Transacción duplicada



Observaciones

- En el flujo exitoso, el débito y el crédito ocurren dentro de una transacción ACID gestionada por ProcesadorTransaccion (C-06); si cualquiera falla, se revierte todo (RNF-01, RB-07).
- En los flujos de rechazo, ningún saldo se modifica: el sistema valida antes de procesar.
- La detección de duplicados aplica idempotencia por referencia única (RB-05): una solicitud repetida devuelve el resultado original sin generar un segundo movimiento.

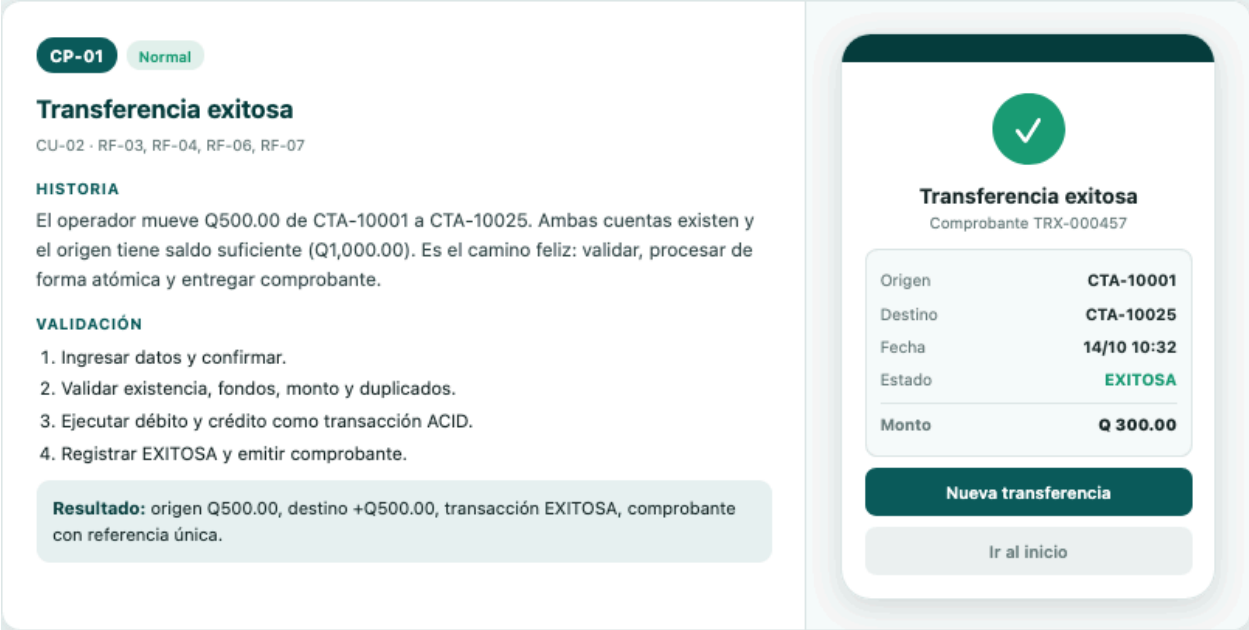
Supuestos del sistema

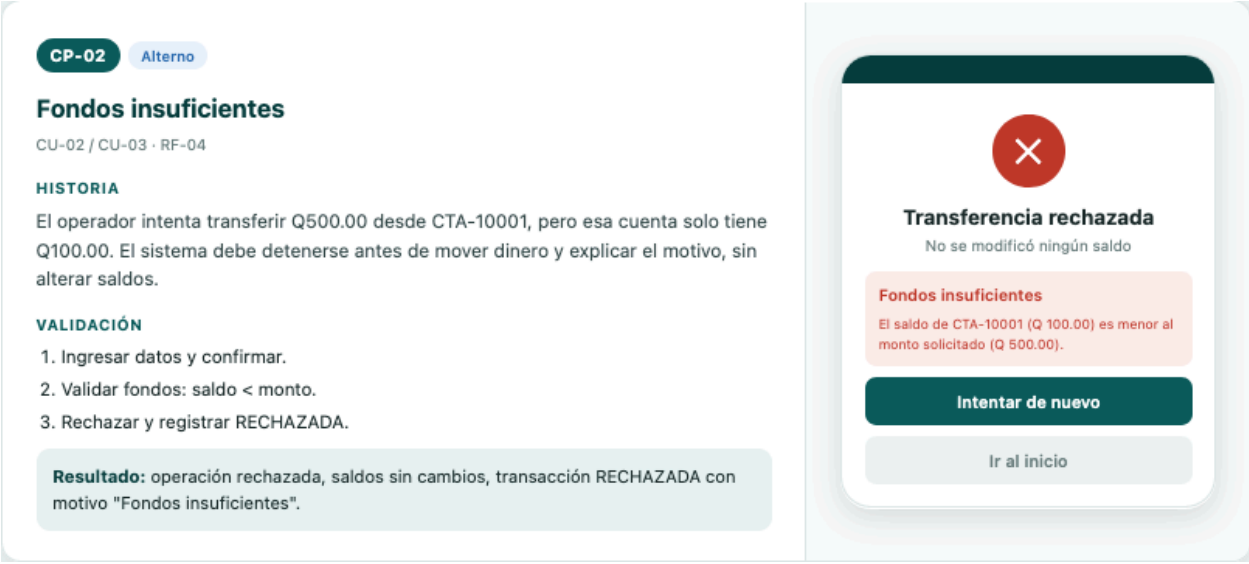
Flujos UX

Wireframes

Casos de Pruebas

Por cada caso de uso, al menos un escenario normal, uno alterno y uno de error. Cada caso cuenta su historia (contexto), define la prueba (pasos y resultado) y muestra un diagrama de flujo que demuestra el camino funcional de la prueba. Cada caso referencia su caso de uso (CU-xx) y los requerimientos funcionales que verifica (RF-xx).





CP-03 Error

Cuenta destino inexistente

CU-02 / CU-03 · RF-04

HISTORIA

El operador escribe por error una cuenta destino que no existe (CTA-99999). El sistema debe rechazar en la validación, antes de tocar saldos, e indicar el problema.

VALIDACIÓN

1. Ingresar destino CTA-99999 y confirmar.
2. Validar existencia: no se encuentra.
3. Rechazar y registrar el intento.

Resultado: mensaje "Cuenta destino no encontrada", sin cambios en saldos, transacción RECHAZADA.



CP-04 Error

Transferencia duplicada

CU-02 / CU-03 · RF-05

HISTORIA

Por un reintento (doble clic o reenvío de red) llega otra vez una transferencia con la misma referencia única (REF-000457) de una operación ya ejecutada. El sistema no debe mover el dinero de nuevo: reconoce el duplicado y devuelve el resultado original. Es la garantía de idempotencia.

VALIDACIÓN

1. Reenviar solicitud con referencia ya procesada.
2. Detectar la referencia repetida.
3. Devolver el resultado original sin re-ejecutar.

Resultado: sin segundo movimiento, se devuelve el resultado original, mensaje "Operación duplicada detectada", saldos sin cambios.



CP-05 Error

Monto inválido

CU-02 / CU-03 · RF-04

HISTORIA

El operador intenta transferir un monto de Q0.00 o negativo. El sistema debe impedirlo en la validación del formulario, antes de procesar nada.

VALIDACIÓN

1. Ingresar monto Q0.00 o negativo y confirmar.
2. Validar monto y rechazar.

Resultado: operación rechazada antes de procesar, mensaje "Monto inválido", saldos sin cambios.



CP-06 Normal

Creación de cuenta exitosa

CU-01 · RF-01

HISTORIA

Un administrador registra una cuenta nueva con datos válidos. No existe otra con el identificador propuesto, así que el sistema la crea con ID único, activa y con saldo inicial en cero.

VALIDACIÓN

1. Ingresar datos y solicitar creación.
2. Validar datos obligatorios y unicidad del ID.
3. Registrar la cuenta y confirmar.

Resultado: cuenta con ID único, estado ACTIVA y saldo Q0.00; creación confirmada.



CP-07 Error

Identificador de cuenta duplicado

CU-01 · RF-01

HISTORIA

El administrador intenta crear una cuenta con un identificador que ya existe (CTA-10025). El sistema debe rechazar la operación y dejar intacta la cuenta original.

VALIDACIÓN

1. Solicitar crear cuenta con ID CTA-10025.
2. Detectar que el identificador ya existe.
3. Rechazar la creación.

Resultado: operación rechazada, la cuenta original sin cambios, mensaje de identificador duplicado.



CP-08 Normal

Consulta de historial con resultados

CU-04 · RF-08

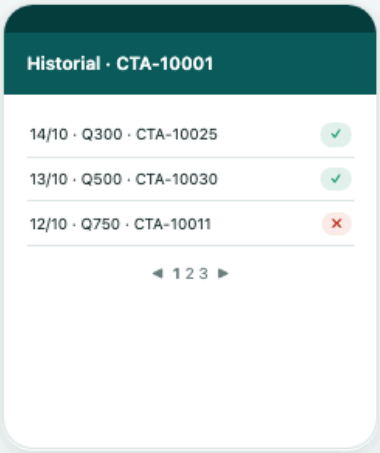
HISTORIA

Un usuario con permisos revisa el historial de CTA-10001, que tiene 3 transacciones. El sistema verifica permisos y muestra la lista, distinguiendo exitosas de rechazadas.

VALIDACIÓN

1. Acceder al historial y filtrar por CTA-10001.
2. Verificar permisos y recuperar transacciones.

Resultado: 3 transacciones con fecha, monto, destino y estado; distingue EXITOSA de RECHAZADA.



CP-09 Alterno

Consulta de historial sin resultados

CU-04 · RF-08

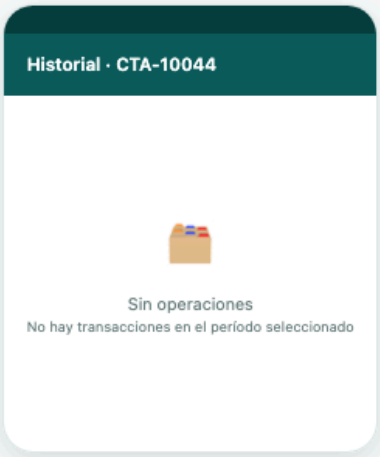
HISTORIA

El usuario consulta el historial de CTA-10044 para un período sin movimientos. El sistema debe responder con un estado vacío claro, sin error ni pantalla en blanco.

VALIDACIÓN

1. Filtrar por CTA-10044 en período sin transacciones.
2. Consultar y no encontrar registros.

Resultado: resultado vacío con mensaje "Sin operaciones"; sin error.



Criterios de Aceptación

CA-01 · Transferencia exitosa

RF: RF-03, RF-06, RF-07 · CP: CP-01

- Dado que la cuenta CTA-10001 tiene saldo de Q1,000.00
- Y la cuenta CTA-10025 existe y está habilitada
- Cuando el usuario solicita transferir Q500.00 de CTA-10001 a CTA-10025
- Entonces el saldo de CTA-10001 debe quedar en Q500.00
- Y el saldo de CTA-10025 debe aumentar en Q500.00
- Y debe registrarse una transacción con estado "EXITOSA"
- Y el usuario debe recibir un comprobante con referencia única

CA-02 · Rechazó por fondos insuficientes

- RF: RF-04 · CP: CP-02
- Dado que la cuenta CTA-10001 tiene saldo de Q100.00
- Cuando el usuario solicita transferir Q500.00 desde CTA-10001
- Entonces la transferencia debe ser rechazada
- Y el saldo de CTA-10001 no debe modificarse
- Y debe registrarse una transacción con estado "RECHAZADA"
- Y el mensaje debe indicar "Fondos insuficientes"

CA-03 · Rechazo por cuenta inexistente

RF: RF-04 · CP: CP-03

- Dado que la cuenta CTA-99999 no existe en el sistema
- Cuando el usuario intenta transferir a CTA-99999
- Entonces la transferencia debe rechazarse antes de modificar cualquier saldo

- Y el sistema debe responder "Cuenta destino no encontrada"
- Y ningún saldo debe modificarse

CA-04 · Prevención de duplicados

RF: RF-05 · CP: CP-04

- Dado que se ejecutó una transferencia con la referencia REF-000457
- Cuando se recibe nuevamente una solicitud con la misma referencia REF-000457
- Entonces el sistema no debe ejecutar una segunda transferencia
- Y debe devolver el resultado original de la operación
- Y el mensaje debe indicar "Operación duplicada detectada"
- Y los saldos no deben modificarse

CA-05 · Rechazo por monto inválido

RF: RF-04 · CP: CP-05

- Dado que las cuentas CTA-10001 y CTA-10025 existen y están habilitadas
- Cuando el usuario solicita transferir un monto de Q0.00 o negativo
- Entonces la transferencia debe rechazarse antes de procesar
- Y el mensaje debe indicar "Monto inválido"
- Y ningún saldo debe modificarse

CA-06 · Creación de cuenta con identificador único

RF: RF-01 · CP: CP-06

- Dado que el administrador proporciona datos válidos para una nueva cuenta
- Y no existe otra cuenta con el mismo identificador
- Cuando se solicita crear la cuenta
- Entonces el sistema debe registrar la cuenta con un identificador único

- Y la cuenta debe quedar activa con saldo inicial de Q0.00
- Y el sistema debe confirmar la creación

CA-07 · Rechazo de identificador duplicado

RF: RF-01 · CP: CP-07

- Dado que ya existe la cuenta CTA-10025
- Cuando se intenta crear otra cuenta con el identificador CTA-10025
- Entonces la operación debe ser rechazada
- Y la cuenta original debe permanecer sin cambios
- Y el sistema debe indicar que el identificador ya existe

CA-08 · Consulta de historial

RF: RF-08 · CP: CP-08

- Dado que la cuenta CTA-10001 tiene 3 transacciones registradas
- Y el usuario tiene permisos de consulta
- Cuando el usuario consulta el historial de CTA-10001
- Entonces el sistema debe mostrar las 3 transacciones
- Y cada transacción debe incluir fecha, monto, cuenta destino y estado
- Y debe distinguir las operaciones EXITOSA de las RECHAZADA

CA-09 · Consulta sin resultados

RF: RF-08 · CP: CP-09

- Dado que la cuenta CTA-10044 no tiene transacciones en el período consultado
- Cuando el usuario consulta su historial para ese período
- Entonces el sistema debe mostrar un resultado vacío
- Y debe informar "Sin operaciones"

- Y no debe producirse ningún error

Trazabilidad

Artefacto integrador del proyecto. Conecta cada requerimiento funcional con su caso de uso, componente, clase de diseño, caso de prueba y criterio de aceptación, cumpliendo la trazabilidad obligatoria.

Matriz principal

RF	Descripción	Regla(s) negocio	Caso de uso	Componente	Clase de diseño	Caso de prueba	Criterio aceptación
RF-01	Crear cuentas	RB-01	CU-01	C-03, C-07	CuentaService, Cuenta, CuentaRepository	CP-06, CP-07	CA-06, CA-07
RF-02	Consultar cuentas	RB-03	CU-01	C-03, C-07	CuentaService, CuentaRepository	CP-06	CA-06
RF-03	Solicitar transferencias	RB-01	CU-02	C-01, C-02	TransferenciaService	CP-01	CA-01
RF-04	Validar transferencias	RB-02, RB-03	CU-02, CU-03	C-05	ValidadorFondos	CP-02, CP-03, CP-05	CA-02, CA-03, CA-05
RF-05	Evitar duplicados	RB-05	CU-02, CU-03	C-05	ValidadorFondos, HistorialRepository	CP-04	CA-04
RF-06	Procesar transferencias	RB-04, RB-07	CU-02	C-06	ProcesadorTransaccion, Cuenta	CP-01	CA-01
RF-07	Registrar transacciones	RB-06	CU-02, CU-03	C-08	HistorialRepository, Transaccion	CP-01	CA-01

RF-08	Consultar historial	RB-06	CU-04	C-04, C-08	ConsultaService, HistorialRepository	CP-08, CP-09	CA-08, CA-09
RF-09	Gestionar errores	RB-06, RB-07	CU-03	C-02, C-05	TransferenciaService, ValidadorFondos	CP-02, CP-03, CP-05	CA-02, CA-03, CA-05

Trazabilidad de requerimientos no funcionales

RNF	**Atributo**	**Se garantiza en (componente/clase)**	**Se verifica en**
RNF-01	Consistencia (atomicidad)	C-06 ProcesadorTransaccion (transacción ACID)	CP-01, CP-02
RNF-02	Confiabilidad	C-02 TransferenciaService (resultado determinable, idempotencia)	CP-04
RNF-03	Seguridad	C-01 API (autenticación/autorización por rol)	CP-08 (permisos)
RNF-04	Integridad y trazabilidad	C-08 HistorialRepository (referencia única + estado)	CP-01, CP-08
RNF-07	Mantenibilidad	C-05 dominio aislado + C-09 anticorrupción	(revisión de diseño)
RNF-08	Privacidad	C-04 ConsultaService (consulta según permisos)	CP-08

Cobertura de casos de uso

Caso de uso	RF cubiertos	Escenarios de prueba	Estado
CU-01 Crear cuenta	RF-01, RF-02	CP-06 (normal), CP-07 (error)	Cubierto
CU-02 Realizar transferencia	RF-03, RF-04, RF-06, RF-07	CP-01 (normal), CP-02 (alterno), CP-03/CP-04/CP-05 (error)	Cubierto
CU-03 Rechazar inválida/duplicada	RF-04, RF-05, RF-09	CP-02, CP-03, CP-04, CP-05	Cubierto
CU-04 Consultar historial	RF-08	CP-08 (normal), CP-09 (alterno)	Cubierto

Verificación de integridad de la trazabilidad

- Todos los RF (RF-01..RF-09) tienen al menos un caso de uso, un componente, una clase, un caso de prueba y un criterio de aceptación asociados.
- Todos los casos de uso (CU-01..CU-04) están cubiertos por escenarios normal, alterno y/o error.
- Todos los componentes (C-01..C-09) aparecen referenciados por al menos un requerimiento.
- Todas las clases de diseño del diagrama de clases están ligadas a un componente y a un requerimiento.
- Todos los casos de prueba (CP-01..CP-09) tienen su criterio de aceptación verificable correspondiente.